

! 练习 5.4.4: 为下面的产生式写出一个和例 5.10 类似的 L 属性 SDD。这里的每个产生式表示一个常见的 C 语言中那样的控制流结构。你可能需要生成一个三地址语句来跳转到某个标号 L, 此时你可以生成语句 goto L。

1) $S \rightarrow \text{if}(C) S_1 \text{ else } S_2$

$L_1 = \text{new}(); L_2 = \text{new}()$

$C.\text{true} = L_1; C.\text{false} = L_2$

$S_1.\text{next} = S.\text{next}; S_2.\text{next} = S.\text{next}$

$S.\text{code} = C.\text{code} || \text{"label"} || L_1 || S_1.\text{code}$

$|| \text{"label"} || L_2 || S_2.\text{code}$

继承属性

练习 5.4.5: 按照例 5.19 的方法, 把在练习 5.4.4 中得到的各个 SDD 转换成一个 SDT。

$S \rightarrow \text{if}(C) \{ L_1 = \text{new}(); L_2 = \text{new}(); C.\text{true} = L_1; C.\text{false} = L_2; S_1.\text{next} = S.\text{next} \}$

$S_1 \text{ else } \{ S_2.\text{next} = S.\text{next} \}$

$S_2 \{ S.\text{code} = C.\text{code} || \text{"label"} || L_1 || S_1.\text{code} || \text{"goto"} || S.\text{next} || \text{"label"} || L_2 || S_2.\text{code} \}$

练习 5.5.1: 按照 5.5.1 节的风格, 将练习 5.4.4 中得到的每个 SDD 实现为递归下降的语法分析器。

```
void check_get_token(string token){
    assert(cur_input == token);
    get_next_token();
}

string S(label next){
    if (cur_input == "if"){
        label L1, L2;
        string C_code, S1_code, S2_code;

        get_next_token();
        check_get_token("(");
        L1 = new_label();
        L2 = new_label();
        C_code = C(L1, L2);
        check_get_token(")");
        S1_code = S(next);
        check_get_token("else");
        S2_code = S(next);
        return C_code || "label" || L1 || S1_code || "goto"
            || next || "label" || L2 || S2_code;
    }
    else{
        /* 其他语句类型 */
    }
}
```

练习 5.5.2: 按照 5.5.2 节的风格, 将练习 5.4.4 中得到的每个 SDD 实现为递归下降的语法分析器。

on-the-fly

```
void check_get_token(string token) {
    assert(cur_input == token);
    get_next_token();
}

void S(label next) {
    label L1, L2;

    if (cur_input == "if") {
        get_next_token();
        check_get_token("(");
        L1 = new();
        L2 = new();
        C(L1, L2);
        check_get_token(")");

        print("label", L1);
        S(next);
        print("goto", next);

        check_get_token("else");
        print("label", L2);
        S(next);
    }
    else {
        /* 其他语句类型 */
    }
}
```

练习 5.5.5: 按照 5.5.4 节的风格, 将练习 5.4.4 中得到的每个 SDD 和一个 LR 语法分析器一起实现。

递归的栈结构:

栈底 0 1 2 3 4 5 6 7 8 9 10
 ? if (X C) Y S1 else Z S2

$\Rightarrow$ $X \rightarrow \epsilon \mid \dots$; $Y \rightarrow \epsilon \mid \dots$; $Z \rightarrow \epsilon \mid \dots$
 产生式 L 属性 $\rightarrow$ S 属性 动作

$X \rightarrow \epsilon$ top += 1 (2 → 3)

stack[top].L1 = new()

stack[top].L2 = new()

(C.true) stack[top].true = stack[top].L1

(C.false) stack[top].false = stack[top].L2

$Y \rightarrow \epsilon$ top += 1 (5 → 6)

(S1.next) stack[top].next = stack[0].next

$Z \rightarrow \epsilon$ top += 1 (8 → 9)

(S2.next) stack[top].next = stack[0].next

$S \rightarrow \text{if}(XC)YS, \text{else } ZS$ code = stack[4].code ||

"label" || stack[3].L1 ||

stack[7].code || "goto" ||

stack[0].next || "label" ||

stack[3].L2 || stack[10].code

top = 1
 stack[top].code = code